Cheatsheet Kompetisi (C++17)

Prioritas: DP > BFS/DFS > Dijkstra/MST/Topo/DSU > pengenalan pola soal > sisanya.
Semua blok kode sudah dikompilasi dengan `g++ -std=c++17` dan diuji. Format edge berbeda antar template, jadi cek signature fungsi sebelum menyalin.

---

0. Setup Kontes

```
#include <bits/stdc++.h>
using namespace std;
typedef long long ll;
typedef pair<int,int> pii;
typedef vector<int> vi;
typedef vector<ll> vll;
#define all(x) (x).begin(),(x).end()
#define sz(x) (int)(x).size()
const ll LINF=(ll)1e18;
const int IINF=1e9;
const int MOD=1e9+7;
void solve(){
}
int main(){
    ios::sync_with_stdio(0);cin.tie(0);
    int t=1;
    cin>>t;
    while(t--)solve();
}
```

Kompilasi untuk debug: `g++ -std=c++17 -O2 -Wall -Wextra -fsanitize=address,undefined -o a a.cpp`. Untuk submit, cukup `-O2`.

Kompleksitas yang cocok berdasarkan constraint

n	target
≤ 10	O(n!)
≤ 20	O(2^n · n), bitmask DP
≤ 100	O(n³) (Floyd)
≤ 5000	O(n²)
≤ 2e5 – 1e6	O(n log n) / O(n)
≤ 1e9	O(log n) / matematika / binary search

Sekitar 1e8 operasi sederhana per detik masih aman.

---

1. DP

Ciri soal DP: "hitung banyaknya cara", "minimum/maksimum biaya", "apakah mungkin", ada pilihan di tiap langkah, submasalah yang tumpang tindih, state space kecil. Prosedur: definisikan `dp[state]` dalam kata-kata → tulis transisi → tentukan base case → tentukan urutan iterasi → tentukan sel jawaban.

1.1 0/1 Knapsack

Tiap item dipakai paling banyak sekali. Loop kapasitas berjalan **turun** pada DP 1D supaya item tidak terpakai dua kali.
Contoh soal: subset sum, partisi menjadi dua bagian sama, nilai maksimum dengan batas berat, "pilih beberapa item dengan total ≤ X".

```
ll knap01(int W,vi&w,vll&v){
    vll dp(W+1,0);
    for(int i=0;i<sz(w);i++)
        for(int c=W;c>=w[i];c--)
```

```
            dp[c]=max(dp[c],dp[c-w[i]]+v[i]);
    return dp[W];
}
bool subsetSum(vi&a,int S){
    vector<bool> dp(S+1,false);
    dp[0]=true;
    for(int x:a)for(int s=S;s>=x;s--)if(dp[s-
x])dp[s]=true;
    return dp[S];
}
```

Nilai besar tapi total berat/nilai kecil → tukar dimensi (`dp[nilai]=berat minimum`).

1.2 Unbounded Knapsack / Coin Change

Item boleh dipakai berulang. Loop kapasitas berjalan **naik**.
Kombinasi (urutan tidak berpengaruh): koin di loop luar, jumlah di loop dalam. **Permutasi (urutan berpengaruh):** jumlah di loop luar, koin di loop dalam.
Contoh soal: jumlah koin minimum, banyak cara membuat kembalian, rod cutting, "stok tak terbatas".

```
int coinMin(vi&c,int S){
    vi dp(S+1,IINF);
    dp[0]=0;
    for(int s=1;s<=S;s++)
        for(int x:c)
            if(x<=s&&dp[s-x]<IINF)dp[s]=min(dp[s],dp[s-
x]+1);
    return dp[S]>=IINF?-1:dp[S];
}
ll coinWays(vi&c,int S){
    vll dp(S+1,0);
    dp[0]=1;
    for(int x:c)for(int s=x;s<=S;s++)dp[s]=(dp[s]+dp[s-
x])%MOD;
    return dp[S];
```

```
}
ll coinPerms(vi&c,int S){
    vll dp(S+1,0);
    dp[0]=1;
    for(int s=1;s<=S;s++)for(int x:c)if(x<=s)dp[s]=
(dp[s]+dp[s-x])%MOD;
    return dp[S];
}
```

1.3 LIS

`dp[i]` = panjang LIS yang berakhir di i. Versi O(n²): `dp[i]=1+max(dp[j])` untuk `j<i, a[j]<a[i]`.
Versi O(n log n): simpan `t[k]` = ekor terkecil dari subsekuens naik dengan panjang k+1; pakai `lower_bound` untuk strict, `upper_bound` untuk non-decreasing.
Contoh soal: LIS / subsekuens non-decreasing terpanjang, hapus minimum agar terurut (n − LIS), rantai/nesting terpanjang (urutkan satu dimensi, LIS di dimensi lain, misalnya envelope, tie diurutkan menurun), jumlah minimum barisan menurun untuk menutup semua elemen = panjang LIS (Dilworth), LCS pada permutasi bisa diubah jadi LIS.

```
int lisN2(vi&a){
    int n=sz(a),r=0;
    vi dp(n,1);
    for(int i=0;i<n;i++){
        for(int j=0;j<i;j++)if(a[j]
<a[i])dp[i]=max(dp[i],dp[j]+1);
        r=max(r,dp[i]);
    }
    return r;
}
int lisStrict(vi&a){
    vi t;
    for(int x:a){
        auto it=lower_bound(all(t),x);
```

```
        if(it==t.end())t.push_back(x);else *it=x;
    }
    return sz(t);
}
int lisNonDec(vi&a){
    vi t;
    for(int x:a){
        auto it=upper_bound(all(t),x);
        if(it==t.end())t.push_back(x);else *it=x;
    }
    return sz(t);
}
vi lisSeq(vi&a){
    int n=sz(a);
    vi tv,ti,par(n,-1);
    for(int i=0;i<n;i++){
        int p=lower_bound(all(tv),a[i])-tv.begin();
        if(p==sz(tv))
{tv.push_back(a[i]);ti.push_back(i);}
        else{tv[p]=a[i];ti[p]=i;}
        par[i]=p?ti[p-1]:-1;
    }
    vi r;
    for(int i=ti.empty()?
-1:ti.back();i!=-1;i=par[i])r.push_back(a[i]);
    reverse(all(r));
    return r;
}
```

1.4 LCS + rekonstruksi

`dp[i][j]` = LCS dari `a[:i]` dan `b[:j]`. Cocok → diagonal +1, tidak cocok → max(atas, kiri). Telusuri balik dari `(n,m)`.
Contoh soal: LCS, insert/delete minimum untuk mengubah A menjadi B (n+m−2·LCS), palindromic subsequence terpanjang (LCS dari s dan reverse(s)), shortest common supersequence (n+m−LCS).

```
string lcs(const string&a,const string&b){
    int n=sz(a),m=sz(b);
    vector<vi> dp(n+1,vi(m+1,0));
    for(int i=1;i<=n;i++)for(int j=1;j<=m;j++)
        dp[i][j]=a[i-1]==b[j-1]?dp[i-1][j-
1]+1:max(dp[i-1][j],dp[i][j-1]);
    string r;
    int i=n,j=m;
    while(i&&j){
        if(a[i-1]==b[j-1]){r+=a[i-1];i--;j--;}
        else if(dp[i-1][j]>=dp[i][j-1])i--;
        else j--;
    }
    reverse(all(r));
    return r;
}
```

1.5 Edit Distance (Levenshtein)

`dp[i][j]` = operasi minimum untuk mengubah `a[:i]` menjadi `b[:j]`. Insert, delete, replace masing-masing berbiaya 1. Base: `dp[i][0]=i`, `dp[0][j]=j`.
Contoh soal: edit minimum antar dua string, jarak pada spell-checker, "operasi minimum untuk mengubah A menjadi B". Biaya berbeda-beda → ubah angka +1. Hanya insert/delete → n+m−2·LCS.

```
int editDist(const string&a,const string&b){
    int n=sz(a),m=sz(b);
    vector<vi> dp(n+1,vi(m+1));
    for(int i=0;i<=n;i++)dp[i][0]=i;
    for(int j=0;j<=m;j++)dp[0][j]=j;
    for(int i=1;i<=n;i++)for(int j=1;j<=m;j++)
        dp[i][j]=min({dp[i-1][j]+1,dp[i][j-1]+1,dp[i-1]
[j-1]+(a[i-1]!=b[j-1])});
    return dp[n][m];
}
```

1.6 Grid DP

`dp[i][j]` dari `dp[i-1][j]` dan `dp[i][j-1]`. Rintangan → 0 cara / INF biaya.
Contoh soal: jumlah path minimum, banyak path dengan rintangan, koin maksimum yang dikumpulkan, persegi terbesar berisi 1 (`dp=1+min(atas,kiri,diagonal)`), soal dua path (state = dua posisi pada langkah yang sama).

```
ll gridPaths(vector<string>&g){
    int n=sz(g),m=sz(g[0]);
    vector<vll> dp(n,vll(m,0));
    if(g[0][0]=='#')return 0;
    dp[0][0]=1;
    for(int i=0;i<n;i++)for(int j=0;j<m;j++){
        if(i==0&&j==0)continue;
        if(g[i][j]=='#'){dp[i][j]=0;continue;}
        if(i)dp[i][j]+=dp[i-1][j];
        if(j)dp[i][j]+=dp[i][j-1];
        dp[i][j]%=MOD;
    }
    return dp[n-1][m-1];
}
ll gridMinSum(vector<vll>&a){
    int n=sz(a),m=sz(a[0]);
    vector<vll> dp(n,vll(m,LINF));
    dp[0][0]=a[0][0];
    for(int i=0;i<n;i++)for(int j=0;j<m;j++){
        if(i)dp[i][j]=min(dp[i][j],dp[i-1][j]+a[i][j]);
        if(j)dp[i][j]=min(dp[i][j],dp[i][j-1]+a[i][j]);
    }
    return dp[n-1][m-1];
}
```

1.7 Bitmask DP (n ≤ 20)

State = himpunan bagian yang disimpan sebagai integer. `m>>i&1` untuk cek, `m|1<<i` untuk set, `m&~(1<<i)` untuk hapus.

Contoh soal: TSP / Hamiltonian path, menugaskan n tugas ke n orang (biaya minimum), partisi item ke grup, "kunjungi semua node", menghitung susunan dengan pasangan terlarang, soal subset gaya SOS.

```
ll tsp(vector<vll>&d){
    int n=sz(d);
    vector<vll> dp(1<<n,vll(n,LINF));
    dp[1][0]=0;
    for(int m=1;m<(1<<n);m++)for(int i=0;i<n;i++){
        if(dp[m][i]>=LINF||!(m>>i&1))continue;
        for(int j=0;j<n;j++)if(!(m>>j&1))
            dp[m|1<<j][j]=min(dp[m|1<<j][j],dp[m]
[i]+d[i][j]);
    }
    ll r=LINF;
    for(int i=0;i<n;i++)if(dp[(1<<n)-1][i]
<LINF)r=min(r,dp[(1<<n)-1][i]+d[i][0]);
    return r;
}
ll assignMin(vector<vll>&c){
    int n=sz(c);
    vll dp(1<<n,LINF);
    dp[0]=0;
    for(int m=0;m<(1<<n);m++){
        if(dp[m]>=LINF)continue;
        int i=__builtin_popcount(m);
        if(i>=n)continue;
        for(int j=0;j<n;j++)if(!
(m>>j&1))dp[m|1<<j]=min(dp[m|1<<j],dp[m]+c[i][j]);
    }
    return dp[(1<<n)-1];
}
```

Enumerasi submask dari `m`: `for(int s=m;s;s=(s-1)&m)`. Total O(3^n).

1.8 Digit DP

Hitung bilangan di `[0,N]` yang memenuhi sifat digit tertentu. State: `(posisi, sifat sejauh ini, tight, started)`. `tight` = prefiks sama persis dengan prefiks N, `started` = angka nol di depan sudah selesai. Jawaban untuk `[L,R]` = `f(R)-f(L-1)`. Reset memo setiap query. Contoh di bawah: jumlah digit = TS (butuh TS ≤ 170).
Contoh soal: bilangan ≤ N dengan jumlah digit S, tidak ada dua digit bersebelahan yang sama, digit D muncul k kali, habis dibagi K (state = sisa bagi), "hitung x di [L,R] sehingga...".

```
string D;
int TS;
ll memo[20][200][2][2];
ll dd(int p,int s,bool tight,bool st){
    if(s>TS)return 0;
    if(p==sz(D))return s==TS;
    ll&r=memo[p][s][tight][st];
    if(r!=-1)return r;
    r=0;
    int lim=tight?D[p]-'0':9;
    for(int
d=0;d<=lim;d++)r+=dd(p+1,s+d,tight&&d==lim,st||d>0);
    return r;
}
ll countDigitSum(ll N,int S){
    D=to_string(N);TS=S;
    memset(memo,-1,sizeof memo);
    return dd(0,0,true,false);
}
```

Jebakan DP

- Inisialisasi `dp` dengan nilai netral yang benar (0 untuk hitungan, INF untuk min, -INF untuk max). INF + biaya bisa overflow → cek `dp[x]<INF` sebelum transisi.
- Arah loop pada DP 1D menentukan 0/1 atau unbounded.

- Hitungan modulo: `%MOD` setelah setiap penjumlahan; pengurangan pakai `(a-b+MOD)%MOD`.
- Kalau state space terlalu besar, cari struktur monoton: binary search, two pointers, greedy.

---

2. Graf

```
vector<vi> g(n);
vector<vector<pair<int,ll>>> wg(n);
g[u].push_back(v);g[v].push_back(u);
wg[u].push_back({v,w});
```

Pakai node berindeks 0; baca edge sebagai `--u,--v`. Sel grid dipetakan ke node `i*m+j`.

2.1 BFS

Jalur terpendek pada graf **tanpa bobot**, atau saat setiap langkah berbiaya sama. Multi-source BFS: masukkan semua sumber dengan `d=0`. 0-1 BFS: deque, edge bobot 0 → depan, bobot 1 → belakang.
Contoh soal: jalur terpendek di labirin, langkah minimum, api/infeksi yang menyebar (multi-source), jarak ke X terdekat untuk semua sel, word ladder, langkah minimum pada graf state implisit (puzzle), langkah kuda.

```
int dx[4]={1,-1,0,0},dy[4]={0,0,1,-1};
vector<vi> bfsGrid(vector<string>&g,int sx,int sy){
    int n=sz(g),m=sz(g[0]);
    vector<vi> d(n,vi(m,-1));
    queue<pii> q;
    d[sx][sy]=0;q.push({sx,sy});
    while(!q.empty()){
        auto [x,y]=q.front();q.pop();
        for(int k=0;k<4;k++){
```

```
            int a=x+dx[k],b=y+dy[k];
            if(a<0||b<0||a>=n||b>=m||g[a][b]=='#'||d[a]
[b]!=-1)continue;
            d[a][b]=d[x][y]+1;
            q.push({a,b});
        }
    }
    return d;
}
vi bfs(int n,vector<vi>&g,vi src){
    vi d(n,-1),par(n,-1);
    queue<int> q;
    for(int s:src){d[s]=0;q.push(s);}
    while(!q.empty()){
        int u=q.front();q.pop();
        for(int v:g[u])if(d[v]<0)
{d[v]=d[u]+1;par[v]=u;q.push(v);}
    }
    return d;
}
vi bfs01(int n,vector<vector<pii>>&g,int s){
    vi d(n,IINF);
    deque<int> q;
    d[s]=0;q.push_back(s);
    while(!q.empty()){
        int u=q.front();q.pop_front();
        for(auto [v,w]:g[u])if(d[u]+w<d[v]){
            d[v]=d[u]+w;
            if(w)q.push_back(v);else q.push_front(v);
        }
    }
    return d;
}
```

Rekonstruksi path: simpan `par[v]=u`, telusuri mundur dari target, lalu reverse.

2.2 DFS / Komponen Terhubung

Dipakai untuk komponen, deteksi siklus, cek bipartit, tree DP, flood fill, menghitung pulau.
Kedalaman rekursi ≈ 1e5+ bisa membuat stack overflow di beberapa judge: pakai BFS atau DFS iteratif untuk graf yang dalam.

```
vector<vi> G;
vi comp;
void dfs(int u,int c){
    comp[u]=c;
    for(int v:G[u])if(comp[v]<0)dfs(v,c);
}
int components(int n){
    comp.assign(n,-1);
    int c=0;
    for(int i=0;i<n;i++)if(comp[i]<0)dfs(i,c++);
    return c;
}
vi color;
bool hasCycleDfs(int u){
    color[u]=1;
    for(int v:G[u]){
        if(color[v]==1)return true;
        if(color[v]==0&&hasCycleDfs(v))return true;
    }
    color[u]=2;
    return false;
}
bool bipartite(int n){
    vi col(n,-1);
    for(int s=0;s<n;s++){
        if(col[s]!=-1)continue;
        col[s]=0;
        queue<int> q;q.push(s);
        while(!q.empty()){
            int u=q.front();q.pop();
            for(int v:G[u]){
                if(col[v]==-1)
{col[v]=col[u]^1;q.push(v);}
```

```
                else if(col[v]==col[u])return false;
            }
        }
    }
    return true;
}
pii farthest(int s,int n){
    vi d(n,-1);
    queue<int> q;
    d[s]=0;q.push(s);
    int last=s;
    while(!q.empty()){
        int u=q.front();q.pop();last=u;
        for(int v:G[u])if(d[v]<0)
{d[v]=d[u]+1;q.push(v);}
    }
    return {last,d[last]};
}
int treeDiameter(int n){
    int a=farthest(0,n).first;
    return farthest(a,n).second;
}
```

Deteksi siklus: berarah → warna (0 belum dikunjungi, 1 di stack, 2 selesai). Tak berarah → edge ke node yang sudah dikunjungi dan bukan parent, atau union DSU gagal. Bipartit ⇔ tidak ada siklus ganjil.

2.3 Topological Sort (Kahn)

Urutkan node sehingga semua edge mengarah ke depan. Hasil kosong = ada siklus.
Contoh soal: penjadwalan tugas dengan prasyarat, urutan mata kuliah, deteksi siklus di graf berarah, **DP pada DAG** (path terpanjang, jumlah path: proses sesuai urutan topo), urutan terkecil secara leksikografis (ganti queue dengan `priority_queue<int,vi,greater<>>`), "apakah ada urutan yang valid".

```
vi topo(int n,vector<vi>&g){
    vi in(n),r;
    for(int u=0;u<n;u++)for(int v:g[u])in[v]++;
    queue<int> q;
    for(int i=0;i<n;i++)if(!in[i])q.push(i);
    while(!q.empty()){
        int u=q.front();q.pop();
        r.push_back(u);
        for(int v:g[u])if(--in[v]==0)q.push(v);
    }
    if(sz(r)<n)r.clear();
    return r;
}
ll dagLongest(int n,vector<vector<pair<int,ll>>>&g){
    vector<vi> h(n);
    for(int
u=0;u<n;u++)for(auto&e:g[u])h[u].push_back(e.first);
    vi o=topo(n,h);
    if(o.empty())return -1;
    vll d(n,0);
    ll r=0;
    for(int u:o){
        r=max(r,d[u]);
        for(auto [v,w]:g[u])d[v]=max(d[v],d[u]+w);
    }
    return r;
}
```

2.4 DSU / Union-Find

Union dan query "satu himpunan?" hampir O(1).
Contoh soal: komponen terhubung saat edge ditambahkan, Kruskal, "apakah A dan B terhubung", hitung grup, deteksi siklus di graf tak berarah, query offline (proses terurut atau **balik penghapusan jadi penambahan**), mengelompokkan item yang ekuivalen, DSU paritas untuk constraint gaya bipartit.

```
struct DSU{
    vi p,s;
    DSU(int n):p(n),s(n,1){iota(all(p),0);}
    int f(int x){return p[x]==x?x:p[x]=f(p[x]);}
    bool u(int a,int b){
        a=f(a);b=f(b);
        if(a==b)return false;
        if(s[a]<s[b])swap(a,b);
        p[b]=a;s[a]+=s[b];
        return true;
    }
    bool same(int a,int b){return f(a)==f(b);}
    int size(int x){return s[f(x)];}
};
```

2.5 Dijkstra

Jalur terpendek dengan bobot **non-negatif**. O((V+E) log V). Jarak bisa melebihi int, jadi pakai `ll`.
Contoh soal: rute termurah, waktu minimum dengan jalan berbobot, **Dijkstra dengan state tambahan**: node = `(vertex, state tambahan)` misalnya `(kota, sisa kupon)`, `(sel, arah)`, `(node, paritas)`; menghitung jumlah shortest path (`cnt[v]+=cnt[u]` saat sama); minimum dari maksimum edge (ubah relax menjadi `max(du,w)`); shortest path dengan fungsi biaya monoton.

```
vll dijkstra(int n,vector<vector<pair<int,ll>>>&g,int
s,vi*par=nullptr){
    vll d(n,LINF);

priority_queue<pair<ll,int>,vector<pair<ll,int>>,greate
r<>> pq;
    d[s]=0;pq.push({0,s});
    while(!pq.empty()){
        auto [du,u]=pq.top();pq.pop();
        if(du>d[u])continue;
        for(auto [v,w]:g[u])if(du+w<d[v]){
```

```
            d[v]=du+w;
            if(par)(*par)[v]=u;
            pq.push({d[v],v});
        }
    }
    return d;
}
```

State tambahan: id = `v*(K+1)+k`, edge ke `(u,k)` atau `(u,k-1)`; jalankan kode yang sama pada graf yang lebih besar. Contoh lengkap ada di bagian 2.9.
Tidak untuk edge negatif → Bellman-Ford.

2.6 MST: Kruskal / Prim

Hubungkan semua node dengan total bobot minimum. Hasilnya punya `n-1` edge, kalau tidak graf tidak terhubung.
Kruskal = urutkan edge + DSU (terbaik jika sudah punya daftar edge). Prim = heap dari satu node awal (terbaik untuk graf padat/adjacency list).
Contoh soal: biaya minimum menghubungkan kota, maximum spanning tree (urutkan menurun), **minimalkan edge terbesar pada sebuah path** (path minimax = path di MST), second-best MST, buang k edge MST terberat untuk membentuk k klaster, "bangun jalan agar semuanya terhubung dengan biaya minimum", alternatif binary search jawaban + DSU.

```
ll kruskal(int n,vector<array<ll,3>>&e){
    sort(all(e));
    DSU d(n);
    ll t=0;int c=0;
    for(auto&[w,a,b]:e)if(d.u(a,b)){t+=w;c++;}
    return c==n-1?t:-1;
}
ll prim(int n,vector<vector<pair<int,ll>>>&g){
    vector<bool> in(n,false);
```

```
priority_queue<pair<ll,int>,vector<pair<ll,int>>,greate
r<>> pq;
    pq.push({0,0});
    ll t=0;int c=0;
    while(!pq.empty()){
        auto [w,u]=pq.top();pq.pop();
        if(in[u])continue;
        in[u]=true;t+=w;c++;
        for(auto [v,x]:g[u])if(!in[v])pq.push({x,v});
    }
    return c==n?t:-1;
}
```

Format edge Kruskal: `{w,a,b}`.

2.7 Bellman-Ford (sekunder)

Edge negatif. Relax semua edge `n-1` kali; perubahan pada putaran ke-n = ada siklus negatif yang terjangkau dari sumber. O(VE). Contoh soal: shortest path dengan bobot negatif, deteksi siklus negatif (arbitrase), difference constraints (`x_v - x_u <= w` → edge `u→v` bobot `w`).

```
bool bellman(int n,vector<array<ll,3>>&e,int s,vll&d){
    d.assign(n,LINF);
    d[s]=0;
    for(int i=0;i<n;i++){
        bool ch=false;
        for(auto&[a,b,w]:e)if(d[a]<LINF&&d[a]+w<d[b])
{d[b]=d[a]+w;ch=true;}
        if(!ch)return true;
        if(i==n-1)return false;
    }
    return true;
}
```

Format edge di sini: `{a,b,w}` (berbeda dengan Kruskal).

2.8 Floyd-Warshall (hanya jika n ≤ ~400)

Shortest path semua pasangan.

```
void floyd(vector<vll>&d){
    int n=sz(d);
    for(int k=0;k<n;k++)for(int i=0;i<n;i++)for(int
j=0;j<n;j++)
        if(d[i][k]<LINF&&d[k][j]<LINF)d[i][j]=min(d[i]
[j],d[i][k]+d[k][j]);
}
```

Inisialisasi `d[i][i]=0`, edge yang tidak ada = `LINF`.

2.9 Soal graf non-standar (prioritas tertinggi: ini soal pengenalan pola)

Soal "non-standar" biasanya tidak menyebut kata graf, node, atau edge. Ceritanya tentang puzzle, jadwal, permainan, array, atau grid. Pekerjaan utamanya: **menerjemahkan cerita menjadi graf**, baru memilih algoritma. Kode algoritmanya sendiri sudah ada di atas; yang sulit adalah modelnya.

2.9.1 Kerangka berpikir (lakukan berurutan)

1. **Apa node-nya?** Kandidat: posisi, sel grid, konfigurasi lengkap (state puzzle), pasangan `(posisi, info tambahan)`, elemen array, nilai bilangan, hari/waktu.
2. **Apa edge-nya?** Satu langkah atau aksi yang boleh dilakukan. Tentukan: berarah atau tidak, berbobot atau tidak, dan apakah bobotnya bisa 0, bisa negatif, atau selalu sama.
3. **Apa yang ditanyakan?** Pilih dari tabel di 2.9.2.
4. **Hitung ukuran.** Jumlah node (termasuk state tambahan) dan jumlah edge. Kalau node × state > ~1e7, graf tidak boleh dibangun eksplisit: buat tetangga secara implisit, atau cari struktur khusus (greedy, binary search, DP).

5. **Pilih algoritma dari jenis bobot.** Sama semua → BFS. Hanya 0/1 → 0-1 BFS. Non-negatif → Dijkstra. Ada negatif → Bellman-Ford. Tidak ada bobot dan yang ditanya hanya konektivitas → DSU/DFS.

Kalau tidak ketemu modelnya, ulangi dari langkah 1 dengan pilihan node yang lain. Banyak soal sulit menjadi mudah hanya dengan mengganti definisi node (misalnya dari "sel" menjadi "sel + arah").

2.9.2 Pertanyaan soal → algoritma

Yang ditanyakan	Algoritma
Langkah/jarak minimum, semua langkah berbiaya sama	BFS
Biaya minimum, bobot 0/1	0-1 BFS
Biaya minimum, bobot non-negatif	Dijkstra
Biaya minimum, ada bobot negatif / deteksi siklus negatif	Bellman-Ford
Apakah A bisa mencapai B / A dan B terhubung	BFS/DFS atau DSU
Jumlah komponen / ukuran grup	DSU atau DFS
Biaya minimum menghubungkan semuanya	MST (Kruskal/Prim)
Minimalkan edge terbesar / maksimalkan edge terkecil pada path	Kruskal sampai terhubung, atau binary search + BFS/DSU
Urutan yang menghormati prasyarat / ada siklus?	Topological sort
Path terpanjang/terpendek di DAG, hitung path di DAG	Topo order + DP

Yang ditanyakan	Algoritma
Bisa dibagi dua kubu tanpa konflik?	Cek bipartit / DSU paritas
Jumlah shortest path	BFS/Dijkstra + `cnt[]`
Kondisi setelah menghapus edge secara berurutan	Proses terbalik dengan DSU
Setiap node punya tepat satu edge keluar	Graf fungsional: cari siklus

2.9.3 Kata kunci di soal → model

Frasa di soal	Kemungkinan model
"langkah minimum", "paling sedikit gerakan", puzzle, permainan	node = state, edge = gerakan, BFS
"paling banyak K kali", "boleh melewati K rintangan", "punya K kupon"	node = `(posisi, sisa K)`
"waktu tercepat", jalan berbobot	Dijkstra
"semua titik saling terhubung dengan biaya minimum"	MST
"bottleneck", "beban terbesar sepanjang jalur sekecil mungkin", "jembatan terendah"	MST / binary search + BFS
"jarak ke yang terdekat dari semua sel"	multi-source BFS
banyak titik awal, satu tujuan	balik arah edge, BFS dari tujuan
"A harus dikerjakan sebelum B"	topological sort
"dua tim", "musuh dari musuh adalah teman", "tidak boleh bersama"	bipartit / DSU paritas

Frasa di soal	Kemungkinan model
"gabungkan", "satu kelompok", "ekuivalen"	DSU
"setiap orang menunjuk tepat satu orang lain"	graf fungsional
"hapus edge satu per satu, berapa komponen?"	DSU offline, proses terbalik
"berapa banyak jalur terpendek"	BFS/Dijkstra + hitung
"apakah ada siklus", "bisa kembali ke awal"	DFS warna / DSU / topo
hubungan `a < b` antar item	DAG, topo + DP

2.9.4 Pola-pola penting, lengkap dengan template

Pola 1. State sebagai node (graf implisit).
Ciri: puzzle, permainan, "ubah A menjadi B dengan operasi tertentu", jumlah state terbatas. Node = konfigurasi lengkap, edge = satu operasi. Jangan bangun adjacency; hasilkan tetangga di dalam BFS dan simpan `visited` di `map` / `set` /hash. Kompleksitas: O(jumlah state × jumlah gerakan). Cek dulu jumlah state, kalau terlalu besar cari cara lain.

```
int bfsStates(ll start,ll goal,function<vector<ll>(ll)>
nxt){
    map<ll,int> d;
    queue<ll> q;
    d[start]=0;q.push(start);
    while(!q.empty()){
        ll u=q.front();q.pop();
        if(u==goal)return d[u];
        for(ll v:nxt(u))if(!d.count(v))
{d[v]=d[u]+1;q.push(v);}
    }
```

```
    return -1;
}
```

Pola 2. State tambahan (node = posisi + info).
Ciri: batasan "paling banyak K", bahan bakar, kunci, paritas langkah, arah terakhir. Node = `(posisi, info)`. Graf efektif membesar K kali, jadi periksa `V·K` dan `E·K` masih muat. BFS jika semua langkah berbiaya sama, Dijkstra jika berbobot.

Contoh A, grid, boleh menghancurkan paling banyak K dinding, langkah minimum:

```
int minStepsBreakWalls(vector<string>&g,int K){
    int n=sz(g),m=sz(g[0]);
    vector<vector<vi>> d(n,vector<vi>(m,vi(K+1,-1)));
    queue<array<int,3>> q;
    d[0][0][0]=0;q.push({0,0,0});
    while(!q.empty()){
        auto [x,y,k]=q.front();q.pop();
        if(x==n-1&&y==m-1)return d[x][y][k];
        for(int t=0;t<4;t++){
            int a=x+dx[t],b=y+dy[t];
            if(a<0||b<0||a>=n||b>=m)continue;
            int nk=k+(g[a][b]=='#');
            if(nk>K||d[a][b][nk]!=-1)continue;
            d[a][b][nk]=d[x][y][k]+1;
            q.push({a,b,nk});
        }
    }
    return -1;
}
```

Contoh B, graf berbobot, boleh membuat paling banyak K edge menjadi gratis, biaya minimum dari `s` ke `t`:

```
ll dijkstraFree(int
n,vector<vector<pair<int,ll>>>&g,int s,int t,int K){
    vector<vll> d(n,vll(K+1,LINF));
```

```
priority_queue<tuple<ll,int,int>,vector<tuple<ll,int,in
t>>,greater<>> pq;
    d[s][0]=0;pq.push({0,s,0});
    while(!pq.empty()){
        auto [du,u,k]=pq.top();pq.pop();
        if(du>d[u][k])continue;
        for(auto [v,w]:g[u]){
            if(du+w<d[v][k]){d[v][k]=du+w;pq.push({d[v]
[k],v,k});}
            if(k<K&&du<d[v][k+1]){d[v]
[k+1]=du;pq.push({du,v,k+1});}
        }
    }
    ll r=LINF;
    for(int k=0;k<=K;k++)r=min(r,d[t][k]);
    return r;
}
```

Pola 3. Bottleneck (minimalkan yang terbesar).
Ciri: "beban terbesar sepanjang jalur harus sekecil mungkin", "tinggi jembatan minimum", "kecepatan minimum yang cukup". Ada dua cara:

- Urutkan edge naik, tambahkan ke DSU satu per satu sampai `s` dan `t` terhubung. Bobot edge terakhir adalah jawabannya (ekuivalen dengan path di MST).
- Binary search pada jawaban X, lalu cek dengan BFS/DSU hanya memakai edge dengan bobot ≤ X. Pakai cara ini kalau syarat sebenarnya lebih rumit dari sekadar bobot edge.

```
ll bottleneck(int n,vector<array<ll,3>> e,int s,int t){
    sort(all(e));
    DSU d(n);
    for(auto&[w,a,b]:e){
        d.u(a,b);
        if(d.same(s,t))return w;
    }
```

```
    return -1;
}
```

Format edge `{w,a,b}`. Mengembalikan -1 jika `s` dan `t` tidak pernah terhubung. Dipakai untuk `s != t`.

Variasi "maksimalkan yang terkecil": urutkan edge **turun** dan lakukan hal yang sama.

Pola 4. Binary search jawaban + BFS (jarak aman).
Ciri: "maksimalkan jarak minimum ke bahaya", "temukan jalur yang paling jauh dari api". Langkah: (1) multi-source BFS dari semua bahaya untuk mendapat `fd[sel]`; (2) binary search `lim`; (3) cek dengan BFS dari awal yang hanya masuk ke sel dengan `fd ≥ lim`. Monoton: kalau `lim` bisa, `lim-1` pasti bisa.

```
int safestPath(vector<string>&g){
    int n=sz(g),m=sz(g[0]);
    vector<vi> fd(n,vi(m,-1));
    queue<pii> q;
    for(int i=0;i<n;i++)for(int j=0;j<m;j++)if(g[i]
[j]=='F'){fd[i][j]=0;q.push({i,j});}
    while(!q.empty()){
        auto [x,y]=q.front();q.pop();
        for(int k=0;k<4;k++){
            int a=x+dx[k],b=y+dy[k];
            if(a<0||b<0||a>=n||b>=m||fd[a]
[b]!=-1)continue;
            fd[a][b]=fd[x][y]+1;q.push({a,b});
        }
    }
    auto ok=[&](int lim){
        if(fd[0][0]<lim)return false;
        vector<vector<bool>> seen(n,vector<bool>
(m,false));
        queue<pii> q2;q2.push({0,0});seen[0][0]=true;
        while(!q2.empty()){
            auto [x,y]=q2.front();q2.pop();
```

```
            if(x==n-1&&y==m-1)return true;
            for(int k=0;k<4;k++){
                int a=x+dx[k],b=y+dy[k];
                if(a<0||b<0||a>=n||b>=m||seen[a]
[b]||fd[a][b]<lim)continue;
                seen[a][b]=true;q2.push({a,b});
            }
        }
        return false;
    };
    int lo=0,hi=n+m;
    while(lo<hi){int mid=
(lo+hi+1)/2;if(ok(mid))lo=mid;else hi=mid-1;}
    return lo;
}
```

Asumsi: awal di `(0,0)`, tujuan di `(n-1,m-1)`, sel `F` adalah sumber bahaya, dan minimal ada satu `F`.

Pola 5. Banyak sumber, atau balik arah.
Ciri: "jarak ke fasilitas terdekat untuk setiap sel", "banyak titik awal tapi satu tujuan". Banyak sumber → masukkan semua sumber ke antrean awal (sudah ada di `bfs(n,g,src)`). Satu tujuan banyak awal → balik arah semua edge, lalu BFS/Dijkstra dari tujuan. Hasilnya jarak dari setiap node **ke** tujuan hanya dengan satu kali jalan.

Pola 6. Jumlah shortest path.
Ciri: "berapa banyak jalur terpendek". Lakukan BFS (atau Dijkstra) dan tambahkan `cnt[v]+=cnt[u]` setiap kali `d[v]==d[u]+1` (atau jarak sama pada Dijkstra). Jangan lupa modulo.

```
pair<vi,vll> countPaths(int n,vector<vi>&g,int s){
    vi d(n,-1);vll c(n,0);
    queue<int> q;
    d[s]=0;c[s]=1;q.push(s);
    while(!q.empty()){
        int u=q.front();q.pop();
```

```
        for(int v:g[u]){
            if(d[v]<0){d[v]=d[u]+1;q.push(v);}
            if(d[v]==d[u]+1)c[v]=(c[v]+c[u])%MOD;
        }
    }
    return {d,c};
}
```

Pola 7. Graf fungsional (setiap node punya tepat satu edge keluar).
Ciri: `nxt[i]` diberikan, "setiap orang menunjuk satu orang", "simulasi berulang x → f(x)". Struktur: tiap komponen punya satu siklus dengan pohon-pohon yang mengarah ke siklus. Pertanyaan umum: node mana yang ada di siklus, berapa langkah sampai masuk siklus, posisi setelah `K` langkah besar (cari siklus lalu pakai modulo panjang siklus, atau binary lifting).

```
vi inCycle(vi&nxt){
    int n=sz(nxt);
    vi vis(n,-1),cyc(n,0);
    for(int s=0;s<n;s++){
        int u=s;
        while(vis[u]==-1){vis[u]=s;u=nxt[u];}
        if(vis[u]==s){
            int v=u;
            do{cyc[v]=1;v=nxt[v];}while(v!=u);
        }
    }
    return cyc;
}
```

Pola 8. Kubu/konflik (bipartit dan DSU paritas).
Ciri: "bagi menjadi dua tim", "pasangan yang tidak boleh satu tim", "musuh dari musuh adalah teman", kendala "sama" atau "beda" antar dua elemen. Kalau semua kendala diketahui di awal, 2-coloring dengan BFS (`bipartite` di 2.2) cukup. Kalau kendala datang satu per satu dan harus dijawab langsung, pakai DSU paritas: simpan paritas (sama/beda) terhadap root.

```
struct ParityDSU{
    vi p,r;
    ParityDSU(int n):p(n),r(n,0){iota(all(p),0);}
    pii f(int x){
        if(p[x]==x)return {x,0};
        auto [root,pr]=f(p[x]);
        r[x]^=pr;p[x]=root;
        return {root,r[x]};
    }
    bool u(int a,int b,int diff){
        auto [ra,pa]=f(a);
        auto [rb,pb]=f(b);
        if(ra==rb)return (pa^pb)==diff;
        p[ra]=rb;r[ra]=pa^pb^diff;
        return true;
    }
};
```

`u(a,b,diff)` menyatakan kendala `warna[a] xor warna[b] == diff` (`0` = sama, `1` = beda). Mengembalikan `false` jika kendala ini bertentangan dengan kendala sebelumnya.

Pola 9. DSU offline dengan penghapusan: proses terbalik.
Ciri: "edge dihapus satu per satu, berapa komponen setelah tiap penghapusan?". DSU tidak bisa menghapus, tetapi bisa menambah. Balik waktu: mulai dari kondisi akhir (semua yang akan dihapus sudah hilang), lalu tambahkan edge dalam urutan terbalik dan catat jawaban.

```
vi componentsAfterDeletions(int
n,vector<pii>&edges,vi&order){
    int m=sz(edges);
    vector<bool> del(m,false);
    for(int id:order)del[id]=true;
    DSU d(n);
    int comp=n;
    for(int i=0;i<m;i++)if(!del[i])comp-
=d.u(edges[i].first,edges[i].second);
```

```
    vi res(sz(order)+1);
    res[sz(order)]=comp;
    for(int k=sz(order)-1;k>=0;k--){
        comp-
=d.u(edges[order[k]].first,edges[order[k]].second);
        res[k]=comp;
    }
    return res;
}
```

`res[k]` = jumlah komponen setelah `k` penghapusan pertama (`res[0]` = sebelum ada yang dihapus). Pola serupa: query offline yang diurutkan berdasarkan bobot, lalu edge ditambahkan sambil menjawab.

Pola 10. Topological sort + DP.
Ciri: hubungan "A sebelum B", tidak ada siklus, ditanya jalur terpanjang/terpendek, jumlah cara, atau nilai optimal sepanjang rantai. Urutan topo menjamin semua prasyarat sudah dihitung sebelum node dipakai. Template ada di 2.3 (`dagLongest`). Kalau yang ditanya "apakah ada siklus" saja, `topo` yang mengembalikan kosong sudah menjawabnya.

Pola 11. Graf tersembunyi di array atau nilai.
Ciri: array `a[i]` dengan syarat antar pasangan indeks, misalnya "boleh pindah dari i ke j jika `a[i]` dan `a[j]` punya faktor persekutuan", "swap hanya antar posisi i dan j tertentu".

- Hubungkan elemen yang memenuhi syarat lalu tanya komponen: DSU. Untuk syarat faktor persekutuan, jangan hubungkan semua pasangan ($O(n^2)$). Hubungkan setiap bilangan dengan **faktor primanya** (pakai `factorize` di 3.2), sehingga tiap prima menjadi node perantara.
- Dalam satu komponen, elemen bisa ditukar bebas: urutkan elemen dalam tiap komponen untuk mendapat urutan terkecil.

- Kalau syarat bersifat jarak (`|i-j|` kecil), edge hanya antar tetangga dekat, atau pakai sliding window.

Pola 12. Grid sebagai graf.
Ciri: sel adalah node, empat tetangga (atau delapan, atau gerakan kuda) adalah edge. Variasi yang sering muncul: sel berbiaya berbeda → Dijkstra; biaya hanya 0/1 (misalnya "putar sakelar" atau "mengikuti panah") → 0-1 BFS; arah penting (kereta, cermin, laser) → node = `(sel, arah)`; teleport → edge tambahan; komponen sel sejenis → flood fill/DSU.

2.9.5 Tabel ringkas pola lain

Pola	Model
Puzzle / state permainan	node = state, edge = gerakan, BFS
Kendala tambahan (bahan bakar, kunci, K lompatan, paritas)	node = `(posisi, info)`, BFS/Dijkstra
"Minimalkan yang terbesar" / bottleneck	MST (Kruskal) atau binary search + BFS/DSU
"Hubungkan semua dengan murah"	MST
Grid dengan biaya sel berbeda	Dijkstra; biaya 0/1 → 0-1 BFS
Banyak sumber, jarak terdekat	multi-source BFS
Tujuan diketahui, banyak awal	balik graf, BFS dari tujuan
Dependensi / urutan	topological sort
Pasangan konflik, 2 tim	cek bipartit / 2-coloring

Pola	Model
Ekuivalensi / penggabungan grup	DSU
Tiap node tepat satu edge keluar	graf fungsional: cari siklus, binary lifting jika banyak query
Konektivitas offline dengan penghapusan	proses terbalik dengan DSU
Relasi `a<b` antar item	DAG, topo + DP
Hitung shortest path	BFS/Dijkstra + `cnt[]`
Pohon, path terpanjang	diameter dengan dua kali BFS
Saling terjangkau (strongly connected)	SCC (Kosaraju/Tarjan), tidak dibahas di sini; kalau n kecil, cek jangkauan dua arah dengan BFS

2.9.6 Latihan memodelkan (cek jawabanmu dengan tabel di atas)

1. *Ada n kota dan m jalan berbobot. Kamu boleh memakai kupon untuk menggratiskan tepat satu jalan. Biaya minimum dari kota 1 ke n?* → Dijkstra dengan node `(kota, kupon sudah dipakai atau belum)` (Pola 2).
2. *Sebuah robot di grid menghadap ke satu arah, tiap langkah boleh maju atau berbelok 90° dengan biaya 1 untuk berbelok. Langkah minimum ke tujuan?* → node `(sel, arah)`, Dijkstra atau 0-1 BFS (Pola 2 dan Pola 12).
3. *n orang, setiap pasangan "musuh" tidak boleh satu tim. Bisa dibagi dua tim?* → bipartit (Pola 8).
4. *Jembatan punya batas tinggi kendaraan. Kendaraan setinggi h ingin pergi dari A ke B. Tinggi maksimum agar bisa sampai?* → bottleneck, urutkan edge turun, DSU (Pola 3).

5. *Kamu punya 1e5 query: "hapus jalan ke-i, berapa kelompok kota sekarang?"* → proses terbalik dengan DSU (Pola 9).
6. *Tiap siswa menunjuk satu teman. Berapa banyak siswa yang berada dalam lingkaran tertutup?* → graf fungsional (Pola 7).

2.9.7 Kesalahan umum pada soal graf non-standar

- Mendefinisikan node terlalu sederhana sehingga informasi yang menentukan jawaban hilang (lupa arah, sisa kupon, atau paritas).
- Menghitung ukuran graf terlalu optimis: `V·K` dan `E·K` harus dihitung sebelum menulis kode.
- Memakai BFS pada graf berbobot, atau Dijkstra pada graf dengan edge negatif.
- Lupa bahwa graf bisa tidak terhubung, punya self-loop, atau edge ganda.
- Memakai DFS rekursif pada graf sedalam 1e5 atau lebih (stack overflow).
- Menyimpan jarak di `int` padahal total bisa melebihi 2e9.

---

3. Algoritma Sekunder

3.1 Binary Search pada Jawaban

Dipakai saat kelayakan **monoton**: kalau jawaban `x` bisa, maka `x+1` juga bisa (atau sebaliknya). Tulis `ok(x)` sebagai pengecekan greedy/BFS/DSU yang sederhana.
Contoh soal: minimalkan maksimum (bagi array menjadi k bagian, load balancing), maksimalkan minimum (tempatkan sapi/router), waktu minimum untuk selesai, radius terkecil yang menutup semuanya, "apakah mungkin dengan batas X". Untuk bilangan real: ulangi 100 kali, bukan `lo<hi`.

```
ll bsMin(ll lo,ll hi,function<bool(ll)> ok){
    while(lo<hi){
        ll m=lo+(hi-lo)/2;
        if(ok(m))hi=m;else lo=m+1;
    }
    return lo;
}
ll bsMax(ll lo,ll hi,function<bool(ll)> ok){
    while(lo<hi){
        ll m=lo+(hi-lo+1)/2;
        if(ok(m))lo=m;else hi=m-1;
    }
    return lo;
}
bool splitOk(vll&a,int k,ll lim){
    int parts=1;ll cur=0;
    for(ll x:a){
        if(x>lim)return false;
        if(cur+x>lim){parts++;cur=x;}else cur+=x;
    }
    return parts<=k;
}
```

Pastikan batas dipilih sehingga jawaban berada di dalam `[lo,hi]`, dan `hi` adalah nilai yang pasti layak.

3.2 Sieve + SPF / Faktorisasi

Contoh soal: daftar prima ≤ N, faktorisasi banyak bilangan, hitung jumlah pembagi, Euler phi, "bilangan dengan sifat faktor prima tertentu". Satu bilangan sampai 1e12: trial division sampai √n.

```
vi spf;
void sieve(int n){
    spf.assign(n+1,0);
    for(int i=2;i<=n;i++)if(!spf[i])
        for(int j=i;j<=n;j+=i)if(!spf[j])spf[j]=i;
}
```

```
vector<pii> factorize(int x){
    vector<pii> r;
    while(x>1){
        int p=spf[x],c=0;
        while(x%p==0){x/=p;c++;}
        r.push_back({p,c});
    }
    return r;
}
vector<ll> factorizeSlow(ll n){
    vector<ll> r;
    for(ll p=2;p*p<=n;p++)while(n%p==0)
{r.push_back(p);n/=p;}
    if(n>1)r.push_back(n);
    return r;
}
```

3.3 Fenwick Tree (BIT)

Update titik dan jumlah prefiks/rentang dalam O(log n).
Contoh soal: jumlah prefiks dinamis, hitung inversi (proses kiri ke kanan, query berapa elemen lebih besar yang sudah dilihat; kompres nilai dulu), hitung elemen lebih kecil di sebelah kanan, jumlah rentang dengan update.

```
struct BIT{
    int n;vll t;
    BIT(int n):n(n),t(n+1,0){}
    void add(int i,ll v){for(i++;i<=n;i+=i&-i)t[i]+=v;}
    ll sum(int i){ll r=0;for(i++;i>0;i-=i&-
i)r+=t[i];return r;}
    ll range(int l,int r){return sum(r)-(l?sum(l-
1):0);}
};
ll inversions(vi a){
    vi s=a;
    sort(all(s));s.erase(unique(all(s)),s.end());
    BIT b(sz(s));
```

```
    ll r=0;
    for(int i=0;i<sz(a);i++){
        int p=lower_bound(all(s),a[i])-s.begin();
        r+=i-b.sum(p);
        b.add(p,1);
    }
    return r;
}
```

3.4 Aritmetika modular dan kombinatorika

Contoh soal: "jawaban modulo 1e9+7", nCr, pangkat, invers, menghitung susunan. Invers dengan Fermat bekerja karena MOD prima.

```
ll pw(ll b,ll e,ll m=MOD){
    ll r=1;b%=m;
    while(e){if(e&1)r=r*b%m;b=b*b%m;e>>=1;}
    return r;
}
ll inv(ll a){return pw(a,MOD-2);}
const int NF=2000005;
ll fct[NF],ifct[NF];
void initFact(){
    fct[0]=1;
    for(int i=1;i<NF;i++)fct[i]=fct[i-1]*i%MOD;
    ifct[NF-1]=inv(fct[NF-1]);
    for(int i=NF-1;i>0;i--)ifct[i-1]=ifct[i]*i%MOD;
}
ll nCr(int n,int r){
    if(r<0||r>n)return 0;
    return fct[n]*ifct[r]%MOD*ifct[n-r]%MOD;
}
```

3.5 Prefix sum, difference array, two pointers

```
vll prefix(vi&a){
    vll p(sz(a)+1,0);
```

```
    for(int i=0;i<sz(a);i++)p[i+1]=p[i]+a[i];
    return p;
}
vector<vll> prefix2D(vector<vi>&a){
    int n=sz(a),m=sz(a[0]);
    vector<vll> p(n+1,vll(m+1,0));
    for(int i=0;i<n;i++)for(int j=0;j<m;j++)
        p[i+1][j+1]=a[i][j]+p[i][j+1]+p[i+1][j]-p[i]
[j];
    return p;
}
ll rect(vector<vll>&p,int r1,int c1,int r2,int c2){
    return p[r2+1][c2+1]-p[r1][c2+1]-p[r2+1][c1]+p[r1]
[c1];
}
int longestWindowSumLE(vi&a,ll S){
    int l=0,best=0;ll cur=0;
    for(int r=0;r<sz(a);r++){
        cur+=a[r];
        while(cur>S)cur-=a[l++];
        best=max(best,r-l+1);
    }
    return best;
}
```

Difference array: `d[l]+=v; d[r+1]-=v;` lalu prefix-sum. Two pointers butuh semua nilai non-negatif atau syarat yang monoton.

---

4. Referensi Cepat STL

vector: `push_back`, `pop_back`, `back()`, `resize(n,x)`, `assign(n,x)`, `insert(it,x)` O(n), `erase(it)` O(n), `vector<vi> g(n,vi(m,0))`. Pakai `vector<bool>` hanya untuk hemat memori (lambat, tidak bisa direferensikan).

string: `substr(pos,len)`, `find(x)` (mengembalikan `npos`), `s+=c`, `stoi/stoll`, `to_string`, `reverse`, `getline(cin,s)` (setelah

`cin>>x` panggil `cin.ignore()` dulu).

set / multiset: terurut, O(log n). `insert`, `erase(value)` (multiset: menghapus SEMUA yang sama; pakai `erase(find(x))` untuk satu), `find`, `count`, `lower_bound(x)` (pertama ≥ x), `upper_bound(x)` (pertama > x), `begin()`, `rbegin()`. Predecessor: `auto it=s.lower_bound(x); if(it!=s.begin()) --it;`. Selalu pakai member `s.lower_bound`, bukan `std::lower_bound` (O(n) pada set).

map: `m[k]` membuat entri default; pakai `count` / `find` untuk pengecekan tanpa efek samping. Iterasi dengan `for(auto& [k,v]:m)`. `lower_bound` bekerja seperti pada set.

unordered_set/map: O(1) rata-rata tapi bisa di-hack atau lambat; pakai `reserve` dan custom hash bila perlu:

```
struct H{size_t operator()(ll x)const{static const ll
R=chrono::steady_clock::now().time_since_epoch().count(
);x+=R+0x9e3779b97f4a7c15;x=
(x^(x>>30))*0xbf58476d1ce4e5b9;x=
(x^(x>>27))*0x94d049bb133111eb;return x^(x>>31);}};
```

Pemakaian: `unordered_map<ll,int,H> m;`.

priority_queue: max-heap secara default. Min-heap: `priority_queue<int,vi,greater<int>>`. Pair dibandingkan dari first lalu second. Tidak ada decrease-key: push duplikat dan lewati entri basi.

deque / queue / stack: `push_front`, `pop_front`, `front`, `back`, bisa diindeks (deque).

bitset: `bitset<N> b; b[i]=1; b.count(); b<<k; b&c;` percepatan knapsack/reachability: `dp|=dp<<w`.

pair / tuple / array: pair dibandingkan secara leksikografis. `array<ll,3>` terurut leksikografis. Structured binding: `auto`

`[a,b]=p;`.

Algoritma (`<algorithm>`, `<numeric>`)

panggilan	arti
`sort(all(v))`, `sort(all(v),greater<>())`	urutkan, O(n log n)
`sort(all(v),[&](int a,int b){return x[a]<x[b];})`	urutan khusus; com (`<`, bukan `<=`)
`stable_sort`	pertahankan urutan
`reverse(all(v))`	balik
`unique(all(v))`	gabung duplikat yar urutkan dulu; `v.erase(unique(`
`lower_bound(all(v),x)`	pertama ≥ x (rentan
`upper_bound(all(v),x)`	pertama > x
`binary_search(all(v),x)`	ada atau tidak?
`*max_element(all(v))`, `min_element`	nilai ekstrem (meng
`accumulate(all(v),0LL)`	jumlah; pakai `0LL`
`partial_sum(all(v),p.begin())`	prefix sum
`iota(all(v),0)`	0,1,2,...
`fill(all(v),x)`, `memset(a,0,sizeof a)` / `-1`	memset hanya untu
`next_permutation(all(v))`	iterasi permutasi (n
`nth_element(v.begin(),v.begin()+k,v.end())`	elemen ke-k terkeci
`rotate`, `swap`, `max({a,b,c})`, `min({a,b,c})`	
`gcd(a,b)`, `lcm(a,b)`	C++17 `<numeric>`

panggilan	arti
`__builtin_popcount(x)`, `__builtin_popcountll`, `__builtin_ctz`, `__builtin_clz`	trik bit
`count(all(v),x)`, `count_if`	
`any_of`, `all_of`, `none_of`	

Kompresi koordinat

```
vi compress(vi&a){
    vi s=a;
    sort(all(s));s.erase(unique(all(s)),s.end());
    vi r(sz(a));
    for(int
i=0;i<sz(a);i++)r[i]=lower_bound(all(s),a[i])-
s.begin();
    return r;
}
```

I/O cepat: `ios::sync_with_stdio(0);cin.tie(0);` dan pakai `'\n'` bukan `endl`. Jangan campur `scanf` dan `cin` setelah itu. Untuk soal interaktif, flush (`cout<<flush`) dan jangan lepas ikatan stream.

---

5. Tips & Trik

Ketepatan

- Overflow: hasil kali dua `int` di atas ~4.6e4 sudah overflow. Tulis `1LL*a*b`. Jumlah 2e5 nilai sebesar 1e9 butuh `ll`. `INF + w` tidak boleh overflow: pakai `1e18` untuk `ll` (menjumlahkan dua INF masih aman sampai 9.2e18, jangan pernah tiga).

- Off-by-one: tentukan setengah terbuka `[l,r)` dan konsisten. Uji kasus terkecil (`n=1`, `n=2`, semua sama, sudah terurut).
- Multi-test: reset semua array/vector/memo global di setiap test. Jangan `memset` array besar per test kalau `t` besar; bersihkan hanya bagian yang dipakai.
- `size()` bertipe unsigned: `v.size()-1` underflow untuk v kosong. Pakai `sz(v)` (int).
- Pembagian integer membulat ke arah nol dan `%` bisa negatif. Mod non-negatif: `((a%m)+m)%m`. Pembagian ceil untuk bilangan positif: `(a+b-1)/b`.
- Jangan membandingkan double dengan `==`; pakai `fabs(a-b)<1e-9`. Utamakan integer.
- Sorting dengan `<=` di comparator menyebabkan crash.
- Menghapus dari container di dalam range-for atas container itu sendiri adalah undefined behavior.
- `operator[]` pada `map` menyisipkan entri. `set.size()` O(1) tapi `distance(begin,it)` O(n).
- Rekursi: kedalaman di atas ~1e5 bisa crash. Ubah ke iteratif atau BFS.

Kecepatan

- Utamakan `vector` dan array daripada `map` bila kunci berupa integer kecil.
- Hindari `endl`, hindari `string +=` di loop yang membuat salinan, kirim container dengan referensi (`vi&`).
- `unordered_map` + `reserve`, atau vector terurut + binary search, bila `map` terlalu lambat.

Strategi menyelesaikan soal

- Baca semua soal dulu, kerjakan yang termudah dulu. Jangan menghabiskan lebih dari ~30–40 menit pada satu soal tanpa ide baru.

- Tulis constraint → tabel kompleksitas di atas → pilih teknik.
- Pikirkan kasus kecil: coba brute force manual pada input kecil untuk menemukan pola atau state DP.
- Pikirkan terbalik (akhir → awal, hapus → tambah, tujuan → sumber).
- "Minimum dari maksimum" atau "maksimum dari minimum" → binary search jawaban.
- "Hitung cara" dengan modulo → DP. "Biaya min/maks" → DP atau shortest path. "Apakah ada" → greedy/DFS/BFS.
- Saat macet di soal graf, definisikan node dan edge secara eksplisit; kalau graf alaminya terlalu besar, tanyakan info tambahan apa yang harus dibawa state.
- Kalau sampel lolos tapi WA: baca ulang soal untuk constraint yang terlewat (distinct? 1-indexed? edge ganda, self-loop, graf tidak terhubung?).

Debugging

```
#ifdef LOCAL
#define dbg(x) cerr<<#x<<" = "<<(x)<<endl
#else
#define dbg(x)
#endif
```

Kompilasi lokal dengan `-DLOCAL`. Stress test: tulis brute force `brute.cpp`, generator `gen.cpp`, lalu:

```
for i in $(seq 1 500); do ./gen $i > in; ./a < in > o1;
./brute < in > o2; cmp -s o1 o2 || { echo diff on $i;
break; }; done
```

Identitas berguna

- Jumlah 1..n = `n(n+1)/2`. Banyak subset = `2^n`. `nC2 = n(n-1)/2`.

- Banyak pembagi n ≤ 1e9 paling banyak 1344; enumerasi pembagi O(√n).
- Pohon: `n-1` edge. Graf terhubung dengan n node butuh ≥ `n-1` edge.
- XOR: `a^a=0`, `a^0=a`; prefix XOR memberi XOR rentang.
- Bit: `x&-x` bit terendah yang menyala, `x&(x-1)` mematikannya, `(x&(x-1))==0` pangkat dua (x>0).
- Floor log2: `63-__builtin_clzll(x)`.

---

6. Pengingat Alur Kerja

Pelajari konsep → implementasi sekali dari ingatan → kerjakan 2–4 soal → catat kesalahan → ulangi tanpa melihat solusi. Simpan buku catatan kesalahan. Sabtu/Minggu: sebelum menulis kode untuk soal apa pun, tulis nama teknik dan kompleksitasnya di kertas; kalau tidak bisa, berarti kamu sedang menebak.